Python for Data Science — Reference Guide

An essential reference covering NumPy, pandas, matplotlib, and scikit-learn for practising data scientists and ML engineers.

1. NumPy Essentials

NumPy is the foundation of the Python data science stack. It provides fast N-dimensional arrays and mathematical operations that run in optimised C.

- np.array([1,2,3]) — create a 1-D array.
- np.zeros((m, n)) / np.ones((m, n)) — create arrays of zeros or ones.
- np.linspace(start, stop, n) — n evenly spaced values.
- arr.reshape(m, n) — reshape without copying data.
- np.dot(A, B) — matrix multiplication.
- arr.mean(), arr.std(), arr.sum(axis=0) — aggregations.
- np.where(condition, x, y) — vectorised conditional.
- np.concatenate([a, b], axis=0) — stack arrays.

2. Pandas Essentials

Pandas provides DataFrame and Series objects for tabular data manipulation. It is the go-to tool for data cleaning and EDA.

- pd.read_csv('file.csv') — load data from CSV.
- df.head(), df.info(), df.describe() — quick inspection.
- df['col'].value_counts() — frequency table.
- df.dropna() / df.fillna(value) — handle missing data.
- df.groupby('col').agg({'price': 'mean'}) — group aggregation.
- df.merge(other, on='key', how='left') — SQL-style join.
- df.pivot_table(values, index, columns, aggfunc) — pivot.
- df.apply(func) — apply a function row- or column-wise.
- pd.get_dummies(df, columns=['cat']) — one-hot encoding.

3. Matplotlib & Seaborn

Matplotlib is the low-level plotting library; Seaborn provides high-level statistical plots built on top of Matplotlib.

Plot type	Matplotlib	Seaborn
Line chart	`plt.plot(x, y)`	`sns.lineplot(data=df)`
Scatter plot	`plt.scatter(x, y)`	`sns.scatterplot(x, y, data=df)`
Histogram	`plt.hist(x, bins=30)`	`sns.histplot(x, kde=True)`
Box plot	`ax.boxplot(data)`	`sns.boxplot(x, y, data=df)`
Heat map	`plt.imshow(matrix)`	`sns.heatmap(corr, annot=True)`
Bar chart	`plt.bar(x, height)`	`sns.barplot(x, y, data=df)`

4. Scikit-learn Workflow

Scikit-learn has a consistent API: all estimators implement fit(), predict(), and transform(). Use Pipelines to chain preprocessing and modelling steps, preventing data leakage.

- Split data: train_test_split(X, y, test_size=0.2, random_state=42).
- Scale features: StandardScaler().fit_transform(X_train).
- Train: model.fit(X_train, y_train).
- Predict: y_pred = model.predict(X_test).
- Evaluate: classification_report(y_test, y_pred).
- Cross-validate: cross_val_score(model, X, y, cv=5).
- Tune: GridSearchCV(model, param_grid, cv=5).fit(X, y).
- Pipeline: Pipeline([('scaler', StandardScaler()), ('clf', LogisticRegression())]).

5. Useful Python Patterns for ML

List comprehensions. [f(x) for x in data if condition] — concise and fast.

Context managers. Use 'with open(...)' for files, 'with torch.no_grad()' for inference.

Dataclasses. Use @dataclass to define clean configuration objects instead of dicts.

Logging. Replace print() with the logging module for production-quality tracing.

Type hints. Annotate function signatures with types for better IDE support and fewer bugs.

argparse / Hydra. Never hardcode hyperparameters — pass them from the command line or config files.

tqdm. Wrap any iterable in tqdm() for a progress bar during long loops.

6. Recommended Libraries by Task

Task	Primary library	Alternative
Data manipulation	pandas	polars (faster for large data)
Numerical computation	NumPy	JAX (GPU-accelerated)
Classical ML	scikit-learn	LightGBM, XGBoost
Deep learning	PyTorch	TensorFlow / Keras
NLP / LLMs	HuggingFace Transformers	LangChain
Visualisation	Matplotlib / Seaborn	Plotly (interactive)
Experiment tracking	MLflow	Weights & Biases
Data versioning	DVC	LakeFS